Hardware Security Module Secure Design

MITRE eCTF 2026
Team **UTEP**
University of Texas at El Paso

Contents

1 Introduction **2**

2 Security Requirements **2**
2.1 Requirement 1 . . . 2
2.2 Requirement 2 . . . 3
2.3 Requirement 3 . . . 3
2.3.1 Authenticating HSMs with Ed25519 digital signatures . . . 4

3 Security Implementations **5**
3.1 Build . . . 5
3.1.1 global secrets . . . 5
3.1.2 Build Environment . . . 6
3.1.3 Build Tools . . . 6
3.1.4 Build Deployment . . . 6
3.2 HSM-to-HSM Communication . . . 7
3.2.1 Securing the UART Protocol . . . 7
3.3 Functional Security Model: Access Control & Metadata Protection . . . 7
3.3.1 Secure Metadata Verification Logic . . . 7
3.3.2 Permission Enforcement and FAT Interaction . . . 8

1 Introduction

The Hardware Security Module (HSM) emulates the sub-component responsible for the storage of files and the transfer of files to other HSM's. Our design is composed of the following components: 2 HSM's (HSM A and HSM B), a management interface, a host computer, the transfer interface, and a permissions group. The host computer is physically connected to each HSM through a USB debugger. The two HSM's are connected via 2 female-to-female jumper cables and communicate through the UART protocol. The following describes the functionality of our HSM design.

- The HSM can communicate with other HSM's given that it is running the `listen` command, and another HSM uses `interrogate` or `receive` with a valid pin
- The HSM can perform file I/O through `read, write, and receive` commands with a valid pin
- Each HSM can support files from multiple groups, but what permissions an HSM has for those groups is determined during the build process.

2 Security Requirements

This section defines the security requirements of our design.

2.1 Requirement 1

An attacker should not be able to perform any file action without a validly provisioned HSM with the permissions to perform that action on files belonging to that group. Specifically:

- The attacker should not be able to **read** files from an HSM without the read permission.
- The attacker should not be able to **create** files without the write permission.
- The attacker should not be able to **receive** files from other HSMs without the receive permission.
- The receive permission also applies to **interrogation**, meaning that the interrogated device should only return metadata about files for which the requesting device has the receive permission.

How we address it:

"The `security.c` file handles permission validation. When the validate_permission function is called with a Group ID and a permission type:

1. It searches for the Group ID in the `global.secrets` file.
2. If the group is present, it retrieves the associated permissions.
3. It returns true only if the requested action (Read, Create, or Receive) is explicitly allowed for that group.

Metadata Protection (Interrogate Fix)

Having all this previous process in mind, to satisfy the requirement that "interrogated devices should only return metadata" for allowed files, we implement **Permission-Based Filtering** in the `interrogate`

command handler. Instead of blindly listing all files in the FAT, the system iterates through the file system and performs a check against the requesting device's provisioned ID and the file's permissions associated with that ID. If the check fails, the file is silently excluded from the response, preventing unauthorized users from accessing file existence metadata.

2.2 Requirement 2

No PIN-protected action should be able to be completed by a user without prior knowledge of the PIN. This includes confidentiality of the PIN. The HSM should not expose information about its PIN to any unauthorized user.

How we address it: The system stores an HMAC-SHA256 tag of the correct PIN inside `secrets.h`, which is flashed onto the board during provisioning. This stored tag is used only for verification against user-provided inputs. Each time a user enters a PIN, the system computes the HMAC-SHA256 of the input using a randomly generated 32-byte `HMAC_KEY` and immediately compares the result to the stored secret tag. At no point does the user have direct access to the secret stored tag or any interface that exposes it. Currently, the system immediately rejects invalid PINs without an artificial delay, prioritizing rapid failure.

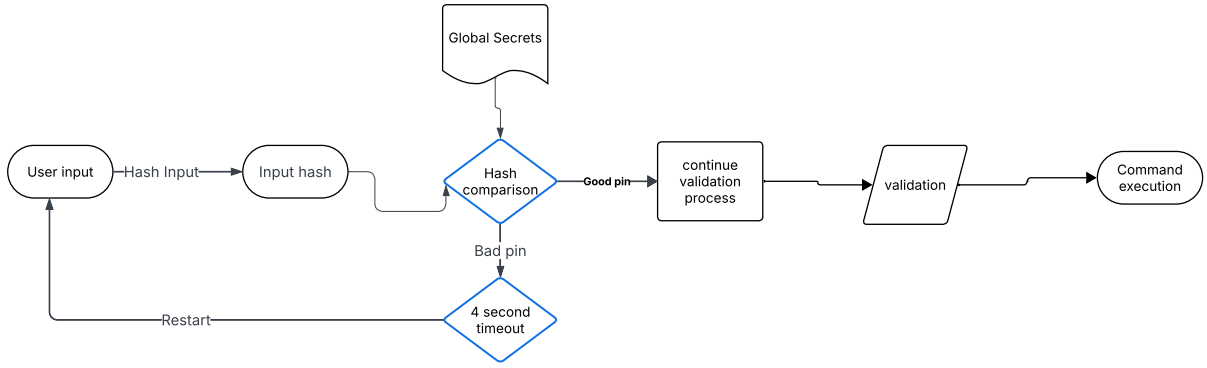

Figure 1: PIN verification and timeout flow.

2.3 Requirement 3

An HSM should not successfully receive any file that was not generated by another valid HSM with write permissions for that group. This includes protecting file integrity from being compromised in any way by an attacker.

How we address it:

- AES-GCM is used to ensure file payload integrity. It generates a 16-byte authentication tag based on the file contents, a slot-specific AES key, and a 12-byte initialization vector (nonce). Upon reading or transferring a file, the HSM relies on the AES-GCM tag stored in flash to ensure the file contents have not been tampered with.
- To verify the sending HSM's receive/write permissions during a transfer, the receiving device checks the permissions sent by the neighbor against its own local permissions lookup using the `validate_permission` function.

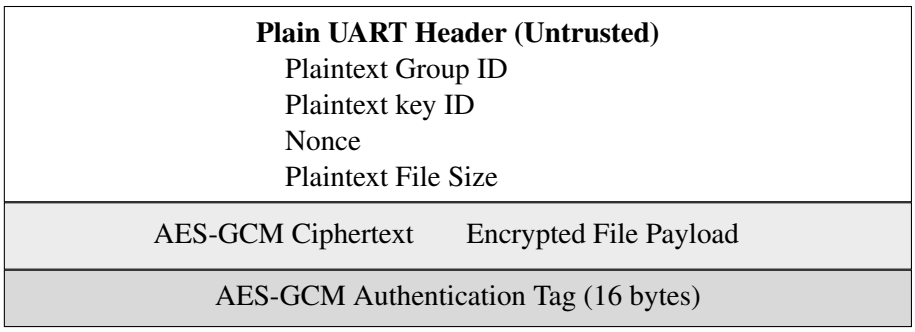

Figure 2: Authenticated Metadata Envelope Packet

2.3.1 Authenticating HSMs with Ed25519 digital signatures

1. The receiving HSM (HSM A) sends a receive request to the sending HSM (HSM B)
2. HSM B receives the request and generates a nonce, **C1** , using the MSPM0L2228's on board TRNG. It then signs the nonce using its ED25519 private key to create **S1** and sends **C1, S1, HSM ID** to HSM A
3. HSM A receives HSM B's response and verifies the source by using ED25519 verify with HSM B's public key that is locally stored on HSM A, if the nonce matches **C1**, HSM A has verified HSM B.

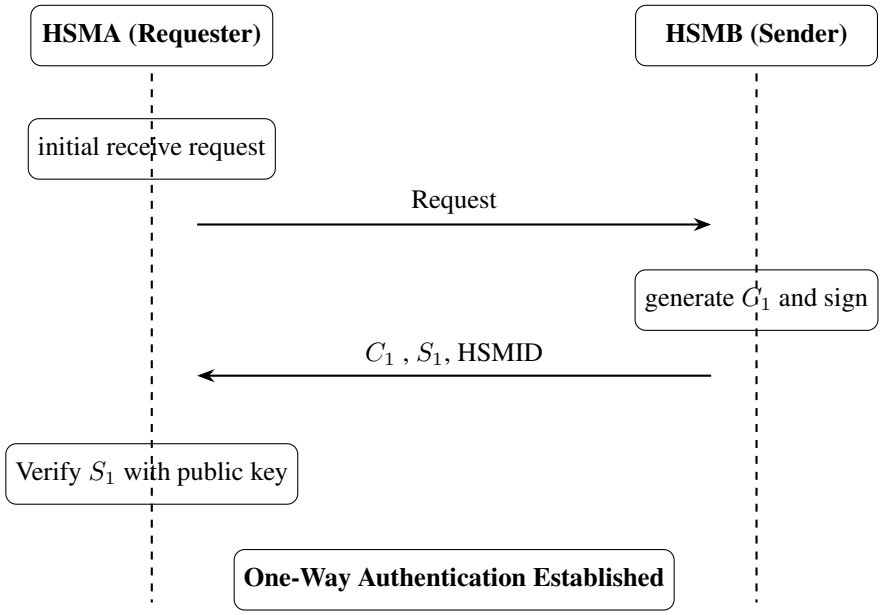

Figure 3: One-Way Authentication Between HSM1 and HSM2 Using Ed25519 Signatures

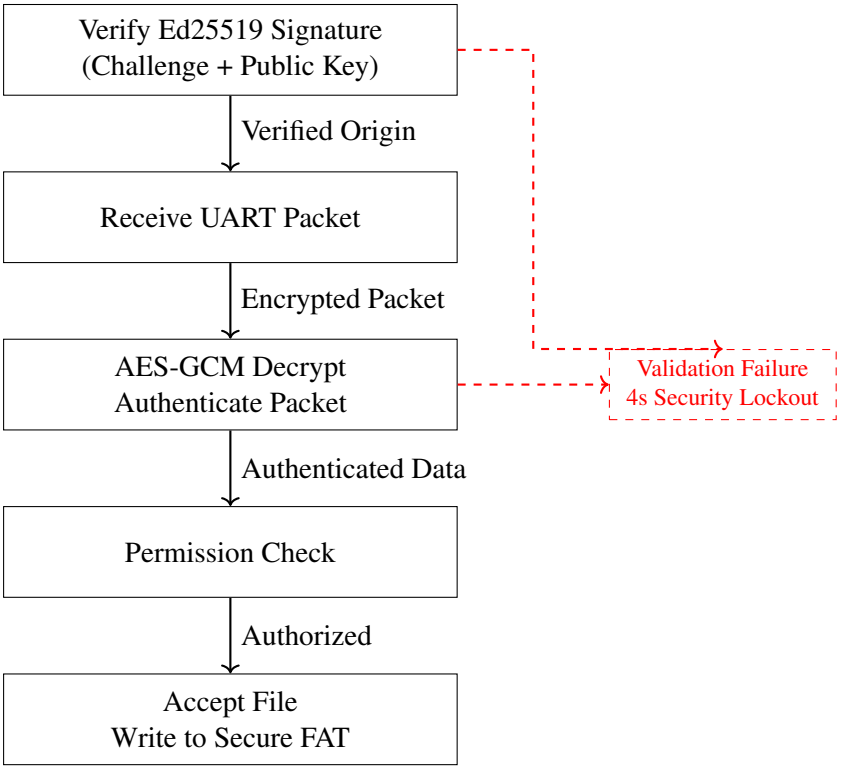

Figure 4: Receiver-Side Validation Pipeline

3 Security Implementations

This section outlines the technical strategies team UTEP is employing to secure the Hardware Security Module(HSM) against adversarial analysis and unauthorized access.

3.1 Build

3.1.1 global secrets

Attackers are **NEVER** given access to the global.secrets file that is generated.

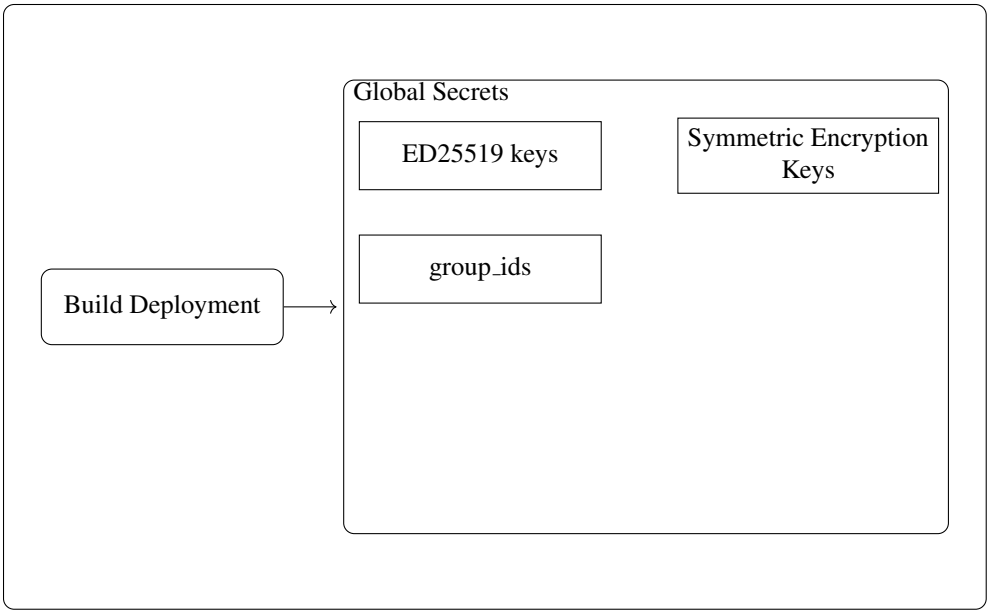

Figure 5: Deployment step generating global secrets

Figure 5 shows the global.secrets file. During the build process, global.secrets is generated with the public keys for use in Ed25519 public key cryptography. In addition, it will contain the symmetric keys needed to encrypt and decrypt all files within the HSM. Finally, it shall contain the group ID's of all groups provisioned to interact with the HSM's.

3.1.2 Build Environment

- Containerized Toolchain: We utilize the competition-provided Docker image that ensures different members can produce byte-identical firmware images no matter what hardware they are using.
- wolfSSL Integration: Unlike standard shared libraries, we will modify ours to include the wolfSSL source tree directly within the firmware directory. This allows us to compile only the necessary cryptographic modules (such as AES-GCM and SHA-256) directly into our binary, minimizing the final size and reducing the attack surface.

3.1.3 Build Tools

Our build process relies on a suite of industry-standard tools optimized for the ARM Cortex-M0+ architecture. The primary compiler is TI-Clang, which is utilized within out Docker container to ensure high-performance code generation and optimal binary density for the MSPM0L2228 micro-controller. We utilize Python3 as the primary scripting engine for our host tools and build-time automation, allowing us to seamlessly bridge the gap between high-level secret management and low-level firmware generation.

3.1.4 Build Deployment

The build deployment phase represents a critical initialization step where the global secrets for an entire run are established. During this stage we use a special script, gen_secrets.py, which generates all necessary cryptographic key material, seeds, and entropy required for the system's secure operation. These generated secrets are stored in a protected global.secrets file, which serves as a read-only input for all subsequent HSM

firmware builds. This architecture ensures that all HSMs within a single deployment share a common root of trust, allowing them to authenticate each other and securely exchange files while remaining isolated from devices in different deployments.

3.2 HSM-to-HSM Communication

To enable secure file transfer between devices, the system utilizes a physical UART link secured by an authenticated encryption scheme. The physical connection is established using **2 female-to-female jumper cables** connecting the UART TX/RX pins between the two HSMs.

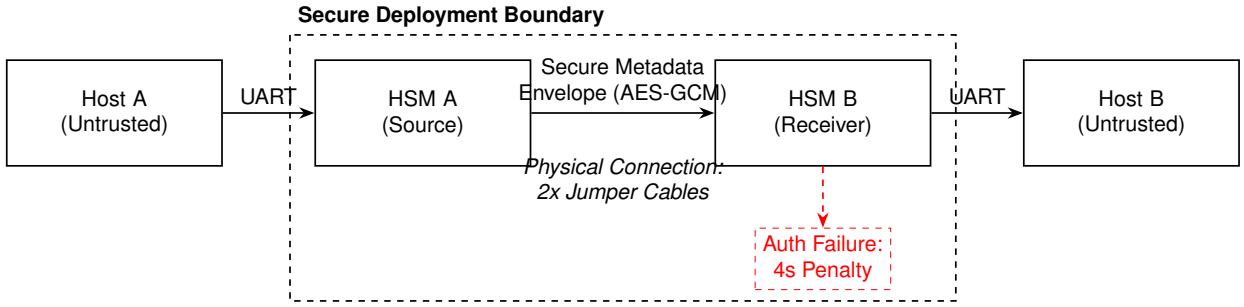

Figure 6: System Logic Model for Secure HSM-to-HSM Communication

3.2.1 Securing the UART Protocol

To protect file contents in transit, we encrypt the file payloads at rest and during transfer.

1. **What is Encrypted:**
   - **Payload Integrity & Confidentiality:** The file content itself is secured using **AES-256-GCM** with a slot-specific key derived from `global.secrets`. This generates both the Ciphertext and a 16-byte Authentication Tag.
   - **Metadata:** File metadata (such as Group ID and File Size) is transferred over UART alongside the encrypted payload to facilitate efficient routing and permission checks.
2. **Validation of Valid HSMs:** The system validates the integrity of the transferred file using shared symmetric keys. Because the slot-specific AES keys are generated during the build and shared only among valid devices in the same deployment, only a legitimate HSM can produce a valid AES-GCM Authentication Tag for the file payload.

3.3 Functional Security Model: Access Control & Metadata Protection

This section models the internal logic of the HSM when processing file-based requests. Our implementation ensures that no action is performed without explicit, cryptographically verified permission, effectively neutralizing unauthorized messages and privilege escalation threats.

3.3.1 Secure Metadata Verification Logic

We implement an ”Envelope” strategy using **Authenticated Encryption (AEAD)**.

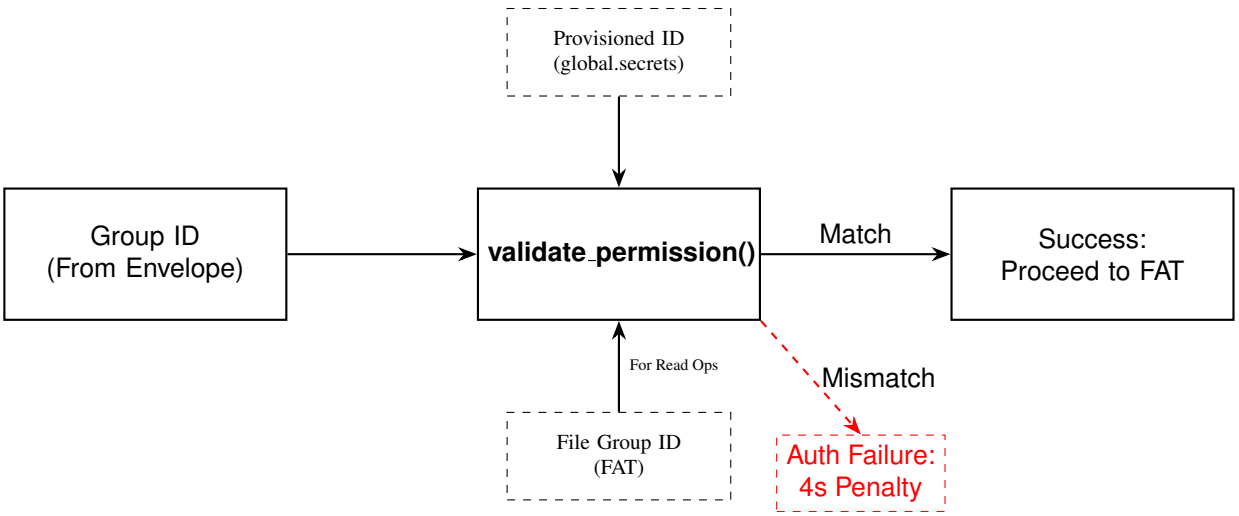

Figure 7: Detailed Logic Model for the `validate_permission` Function

- **Cryptographic Unwrapping:** When a command arrives via UART, the system identifies the Metadata Envelope. The HSM utilizes the `wolfSSL_GCM_Decrypt` function with a 256-bit session key derived from *global.secrets* to verify the 16-byte Authentication Tag.
- **Fail-Fast Mechanism:** If the GCM tag verification fails, the HSM immediately triggers a `SECURITY_FAULT` and initiates a 4-second lockout.

3.3.2 Permission Enforcement and FAT Interaction

Once the metadata is authenticated, the HSM performs a permission check to satisfy **Requirement 1**.

- **Authoritative ID Retrieval:** The `validate_permission()` function retrieves the HSM's immutable **Provisioned Group ID**.
- **GroupID Comparison:** The HSM compares the decrypted Group ID extracted from the Metadata Envelope against the provisioned list in global.secrets. For operations involving existing files (e.g., Read), the system strictly enforces that this decrypted ID matches the Group ID stored in the FAT, ensuring the request corresponds to the actual file classification.
- **Constant-Time Execution:** The comparison logic executes in a fixed number of clock cycles to prevent timing side-channel attacks.

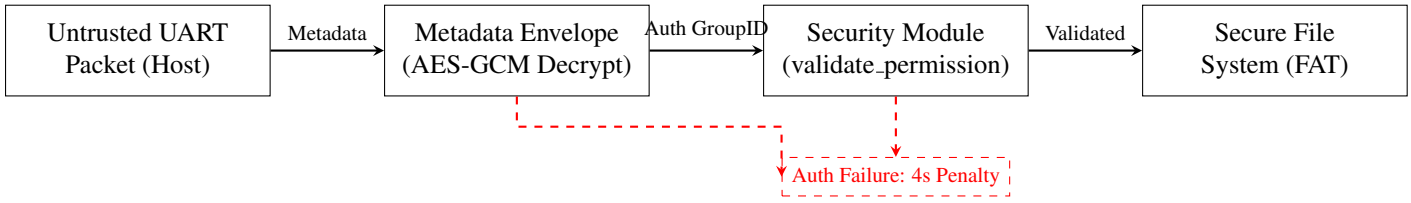

Figure 8: System Logic Model for Secure File Access (Horizontal Architecture).